PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IEE2714-1

Tarea 3

Fecha de entrega:
30 de Octubre De 2026
Estudiante:
Renato Salinas

Link github repository:

https://github.com/The2troller/imagenes/tree/main/tarea%203

Explicaciones:

Pregunta 1:

Una perturbación sinusoidal real provoca componentes en posiciones simétricas del espectro debido a que, como podemos apreciar en base a la formula de las funciones trigonométricas como por ejemplo seno ($sin(wt) = \frac{1}{2}(e^{jwt} - e^{-jwt})$) y coseno ($cos(wt) = \frac{1}{2}(e^{jwt} + e^{-jwt})$). C
Como se mencionó, esta función contiene 2 euler, uno elevado positivamente y otro negativamente en la misma magnitud, por lo tanto su transformada de Fourier exhibirá una simetría respecto al origen. Dentro del código esta característica se realiza de forma literal al aplicar las dos distancias simultáneamente dentro de la misma iteración, al interior de la creación del notch, siendo este el ideal o el gaussiano.
Las coordenadas de frecuencia se representan mediante una función de numpy llamada ogrid, la cual básicamente permite mapear los píxeles siguiendo unas reglas que establecemos, en este caso fueron establecidas considerando que el centro de la imagen debe corresponder a (0,0), de esta manera los cálculos se facilitan. Para cada píxel de espectro la forma en la cual se calcula su distancia al centro de un notch cambia en base al modo utilizado, siendo este el notch ideal o el Gaussiano. Si se usa el filtro ideal la distancia se calcula siguiendo el teorema de pitágoras $d = \sqrt{x^2 + y^2}$, para finalmente a todos los píxeles con una distancia menor o igual que la variable d0, modificarlos a cero. En el caso de gauss, se aplica un cálculo de distancia usando una fórmula de la campana de gauss, y a está se le resta a 1, para así obtener un degradado el cual parte desde 0 y va disminuyendo el que resta hasta valores despreciables, según la curva.
El padding y recorte se implementan mediante la función de numpy pad, los valores generados fuera de esta dependen del modo seleccionado. El modo predeterminado es “symmetric”, el cual simplemente refleja la imagen como si el espejo estuviera en el borde. Cada borde se extiende una cantidad determinada por la variable size, la cual si la dejan en predeterminado sería la mitad del alto y ancho de la imagen original, para así obtener un padding del doble de tamaño de la imagen original, para volver al tamaño original, posteriormente se aplica una división, en donde se recorta, por medio de índices, las posiciones donde debiera estar la imagen original.

El filtro se aplica directamente sobre el espectro complejo y no separadamente sobre sus imágenes por temas de complejidad, sería mucho más complejo separarlos para multiplicarlos, ademas de inutil, ya que al multiplicar un complejo por un número real, simplemente alteramos la magnitud, no la fase, por lo que matemáticamente es irrelevante el multiplicarla por separado, ya que llegamos al mismo resultado.
Usando el notch gauss y eligiendo un pixel cercano al primer notch desde izquierda a derecha, exactamente el ubicado a lo largo del eje x en la posición 120, podemos observar que su valor inicial es de 400.66 + 4784.11j aproximadamente, los cuales son alterados según un H(u,v) de valor 0.625 aproximadamente para obtener 250.23 + 2988.59j
Si se eliminase tan solo uno de ambos pares simétricos, provocaría que la imagen no perdiera el ruido, ya que no se terminaría por eliminar completamente la componente imaginaria dentro de la ecuación. Se aprecia en la figura 4 una comparativa entre una imagen filtrada normalmente y una sin eliminar simétricamente.
Las posiciones de notches fueron seleccionadas en base a el ruido periodico observado dentro del espectro de magnitud de la imagen el cual, como se aprecia en la figura 1, corresponde a un visible punto de energía irregular, cuya totalidad de puntos se encuentran dentro del eje x, y se observó una distancia entre ellos común de 282 píxeles, por lo que se calculó puntos comenzando desde el primero en la posición 141 hasta antes de llegar al centro de la imagen cada 282 píxeles.
Se exploró anchos de notchs variados en ambas configuraciones de notchs (figura 2), y se llegó a la conclusión de que en un notch de d0 = 1, se pierde casi nula información y se puede observar que al restar la imagen original con la transformada únicamente queda el ruido vertical, que es lo que necesitamos. Al aumentar d0 se observa que cada vez se comienza a perder más información, comenzando por el objeto principal.
Al comparar el filtrado sin padding con padding “symmetric” (figura 3) usando dos tamaños de extensión para este último. Se logra apreciar, mediante el uso de recortes ampliados, que el uso de padding es absolutamente necesario para poder eliminar correctamente el ruido dentro de la imagen, ya que al no tenerlo presente la imagen transformada asume una periodicidad circular, en otras palabras, es como si la imagen estuviera pegada a sí misma infinitamente, y en los bordes ocurren colisiones de valores muy distintos, lo que provoca ruidos residuales en el espectro, que finalmente logra que en los bordes de la imagen existan distorsiones. En el caso del tamaño 50x50 contra el H/2xL/2, el primero es insuficiente para el tamaño de esta imagen, por lo que las distorsiones logran atravesar este pequeño margen y terminan contaminando los bordes de la foto original, sin embargo el segundo cumple con la condición exacta para que la transformada de fourier procese la imagen como una convolución lineal perfecta en vez de una circular.
Por todas estas razones, la conclusión final a la que se llega son que los valores de configuración con un mejor resultado son: size = (H/2, L/2), pad_mode = “symmetric” y un valor de d0 = 1. Considerar que esto es para este caso en específico, y el valor de d0 probablemente cambie en relación a la información presentada, ya que al tener valores tan precisos y

conocidos podemos aplicar un valor de d0 tan bajo, sin embargo al tener menor información, el usar un valor tan bajo puede no captar totalmente el ruido a eliminar y dejar de ser eficaz.

Pregunta 2:

La inversión directa Y/H puede ser inestable debido a que la imagen observada es el resultado de multiplicar en frecuencia la imagen original por H y luego sumarle el ruido, es decir $Y = H \cdot X + N$, por lo que al dividir por H no obtenemos solamente X, sino que $X + N/H$, y es ese segundo término el que genera el problema. H es la transformada de una PSF, que básicamente es un filtro pasa bajos, por lo que en las frecuencias altas el valor de |H| se acerca muchísimo a cero, mientras que el ruido gaussiano agregado es ruido blanco, es decir, tiene más o menos la misma magnitud en todas las frecuencias. Por lo tanto, en esas frecuencias estamos dividiendo un ruido que sigue presente por un número casi nulo, lo que lo amplifica enormemente, y como la imagen original ya tenía muy poca energía ahí, lo que termina quedando es básicamente solo ruido de alta frecuencia que tapa la imagen. El término K evita esto debido a que dentro de la función Wiener, el filtro se construye como $W = \text{conj}(H) / (|H|^2 + K)$, por lo que el denominador nunca puede ser menor a K. Cuando $|H|^2$ es mucho mayor que K el filtro se comporta casi igual que 1/H y se invierte el desenfoque, pero cuando $|H|^2$ es mucho menor que K el filtro queda aproximadamente como conj(H)/K, que tiende a cero, por lo que esas frecuencias simplemente se apagan en vez de amplificarse.
La PSF espacial es una matriz pequeña, mientras que la fft2 de la imagen tiene el tamaño completo de la imagen, por lo que para poder multiplicarlas se debe transformar en H(u,v) en tres pasos que se realizan dentro de la función generate_psf. Primero se crea una matriz de ceros del mismo tamaño que la imagen y se copia la PSF dentro de ella, que corresponde al ajuste de tamaño. Después, se desplaza circularmente la PSF para que su centro quede en la posición (0,0) de la matriz, ya que la fft2 considera el píxel (0,0) como el origen. Finalmente se le aplica la fft2 y se guarda en self.h. Si este paso se hiciera de forma incorrecta ocurriría que si no se ajusta el tamaño no se podría multiplicar H con la fft2 de la imagen porque las dimensiones no coinciden, y si no se centrara el origen H tendría una fase lineal adicional, que se traduce en que la imagen desenfocada aparece trasladada circularmente tantos píxeles como lo alejado que estaba el centro de la PSF de la esquina.
El gradiente que se implementó para Tikhonov proviene de derivar la función de costo del enunciado. El término de fidelidad es Ht_diff, que se obtiene de calcular Hx aplicando la fft2 a x, multiplicando por self.h y volviendo con la ifft2, luego se calcula diff = Hx − y, que es el residuo, y finalmente se le aplica Ht para devolverlo al dominio de la imagen. El término de regularización por otro lado es lam * x, el cual penaliza imágenes con mucha energía y evita que la solución crezca amplificando el ruido.
Ht se implementa dentro de la función tikhonov en Ht_diff = np.real(ifft2(fft2(diff) * np.conj(self.h))), es decir, se hace exactamente lo mismo que para H (fft2, multiplicación y ifft2), pero multiplicando por el conjugado de self.h en vez de por self.h. Esto funciona debido a que al

usar convolución circular, aplicar H equivale a multiplicar por H(u,v) en frecuencia, y como la PSF es real, su transpuesta equivale a multiplicar por el conjugado, que en el espacio corresponde a convolucionar con la PSF volteada respecto a su origen. Si la PSF fuera simétrica, como una gaussiana centrada, la PSF volteada sería idéntica a la original, H sería real y por lo tanto Ht = H. Si la PSF no fuera simétrica, por ejemplo un desenfoque de movimiento, H sería compleja y su conjugado sería distinto a H, por lo que en nuestro código no habría que cambiar nada, ya que se usa np.conj.
El criterio de detención utilizado calcula el cambio relativo entre dos iteraciones consecutivas, es decir, la norma de la diferencia entre iteración k+1 y iteración k, dividida por la norma de x más 1e-8 para evitar dividir por cero. La decisión de detenerse se toma dentro del ciclo for de la función tikhonov, donde si el cambio es menor a la tolerancia se imprime el mensaje de convergencia, se asigna x = x_next y se ejecuta el break. Además existe el máximo de iteraciones max_iter, que funciona como respaldo en caso de que el criterio nunca se cumpla. No se utilizó el RMSE como criterio debido a que requiere conocer la imagen ideal, algo que en un caso real no se tendría, por lo que solo se guarda en la lista rmses para poder analizar los resultados.
Para λ = K, la solución cerrada de Tikhonov se obtiene igualando a cero el gradiente de la función de costo, lo que entrega Ht(Hx − y) + λx = 0, y ordenando queda (Ht * H + λI)x = Ht * y. Al usar convolución circular, la transformada de Fourier diagonaliza a H, es decir, aplicar H se convierte en multiplicar por H(u,v) y aplicar Ht en multiplicar por conj(H(u,v)), por lo que la ecuación anterior se separa en una ecuación independiente por cada frecuencia, (|H|² + λ)·$\hat{X}$ = conj(H)·Y, de donde $\hat{X}$ = conj(H)·Y / (|H|² + λ), que es exactamente la misma expresión implementada en la función wiener con K = λ. Con respecto a los detalles que pueden impedir una coincidencia exacta, el principal es que el método iterativo no necesariamente llegó al mínimo, ya que se detuvo por el criterio de tolerancia o por max_iter antes de converger completamente, sobre todo en las frecuencias altas cuando λ es pequeño.

Experimentación y análisis:

Pregunta 1:

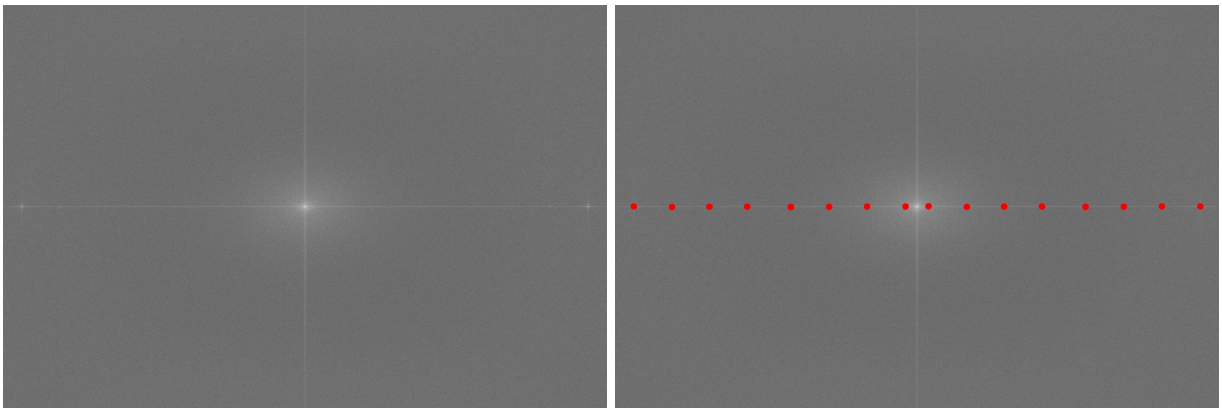


Fig 1. Transformada de Fourier imagen base e imagen con puntos marcando ruido.

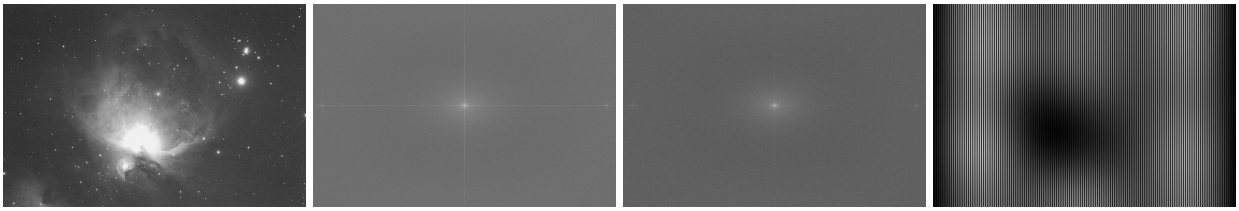


Fig 2.1. Para ideal con d0 = 5, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

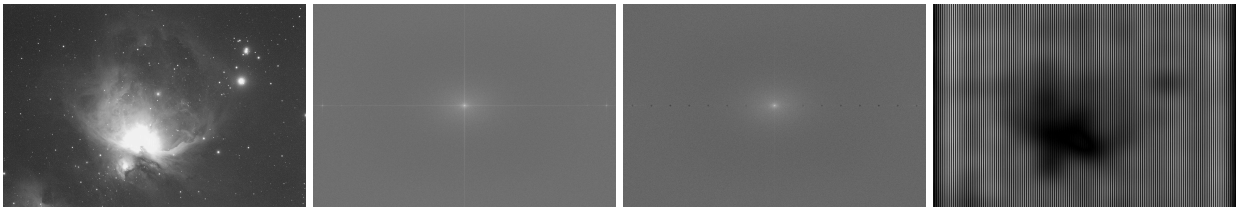


Fig 2.2. Para ideal con d0 = 15, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

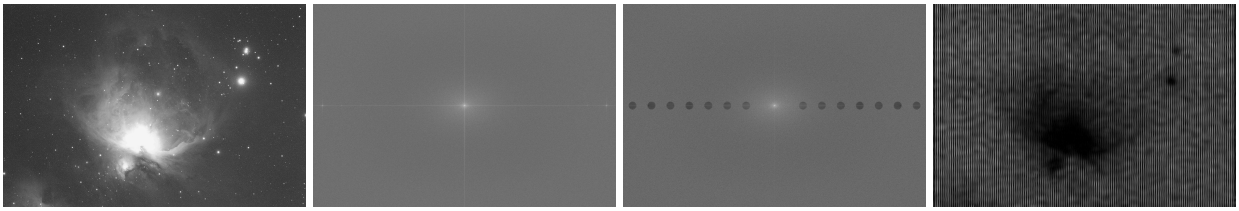


Fig 2.3. Para ideal con d0 = 60, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

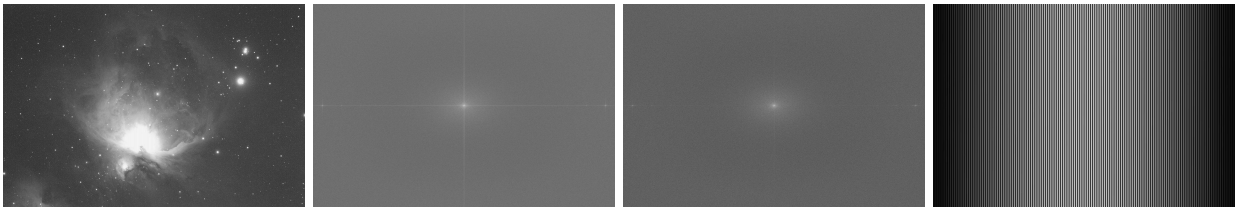


Fig 2.4. Para ideal con d0 = 1, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

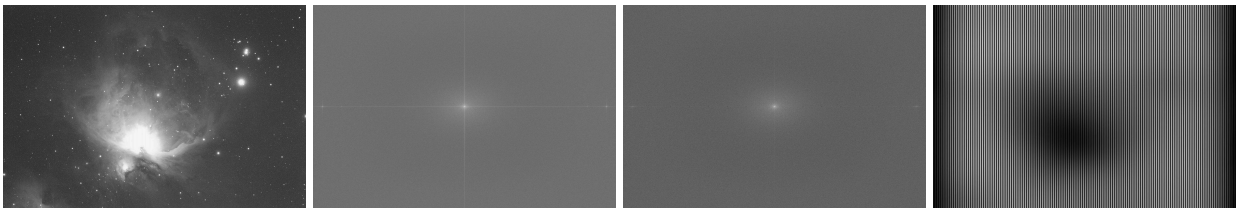


Fig 2.5. Para gauss con d0 = 5, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

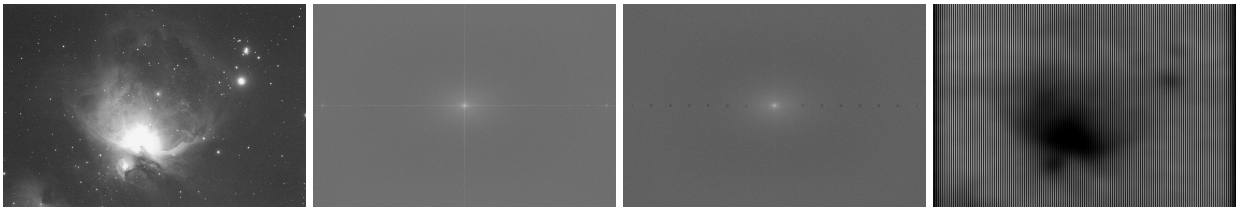


Fig 2.6. Para gauss con d0 = 15, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

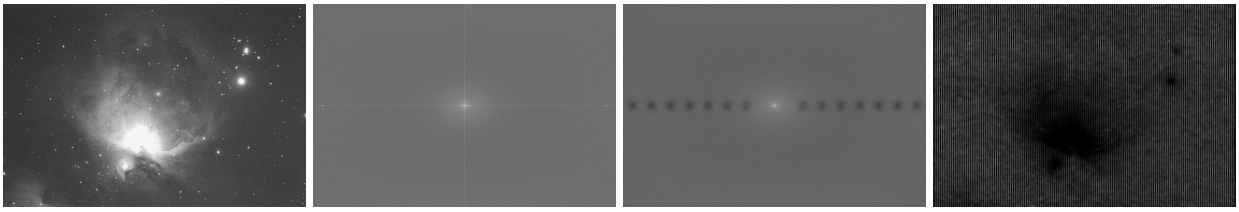


Fig 2.7. Para gauss con d0 = 60, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

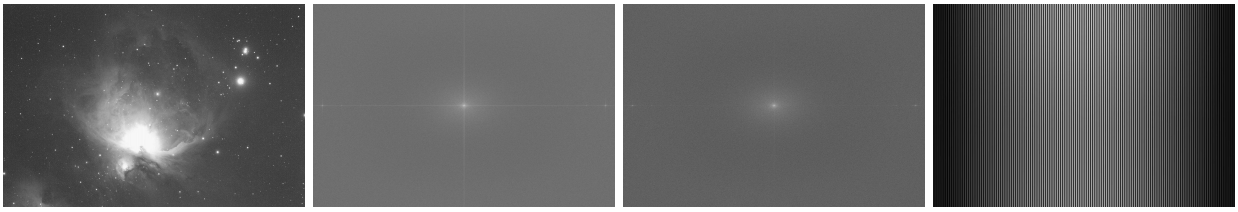


Fig 2.8. Para gauss con d0 = 1, imagen filtrada / espectro antiguo / espectro post / imagen de diferencia con original

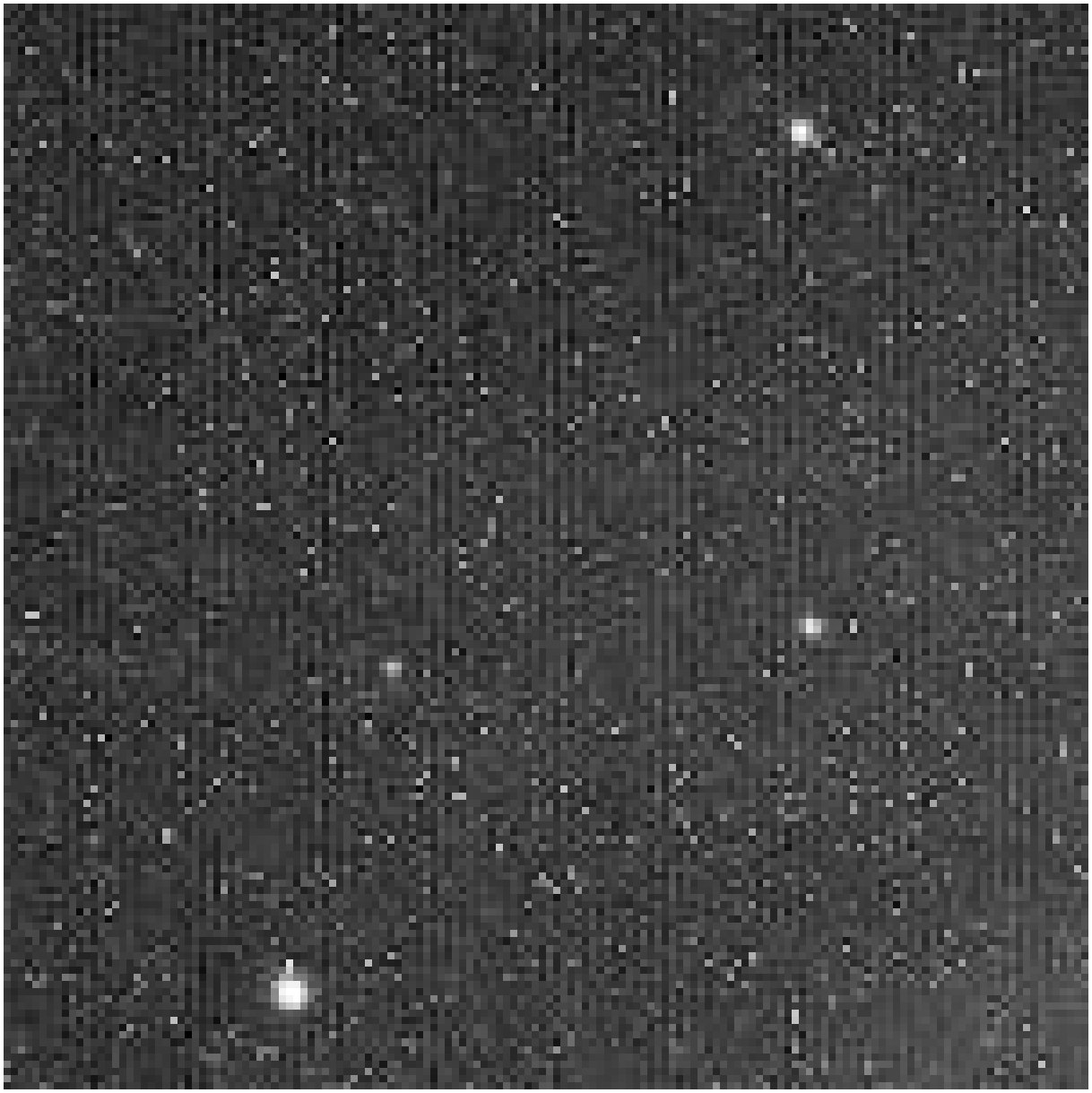


Fig 3.1. Zoom imagen ideal sin padding

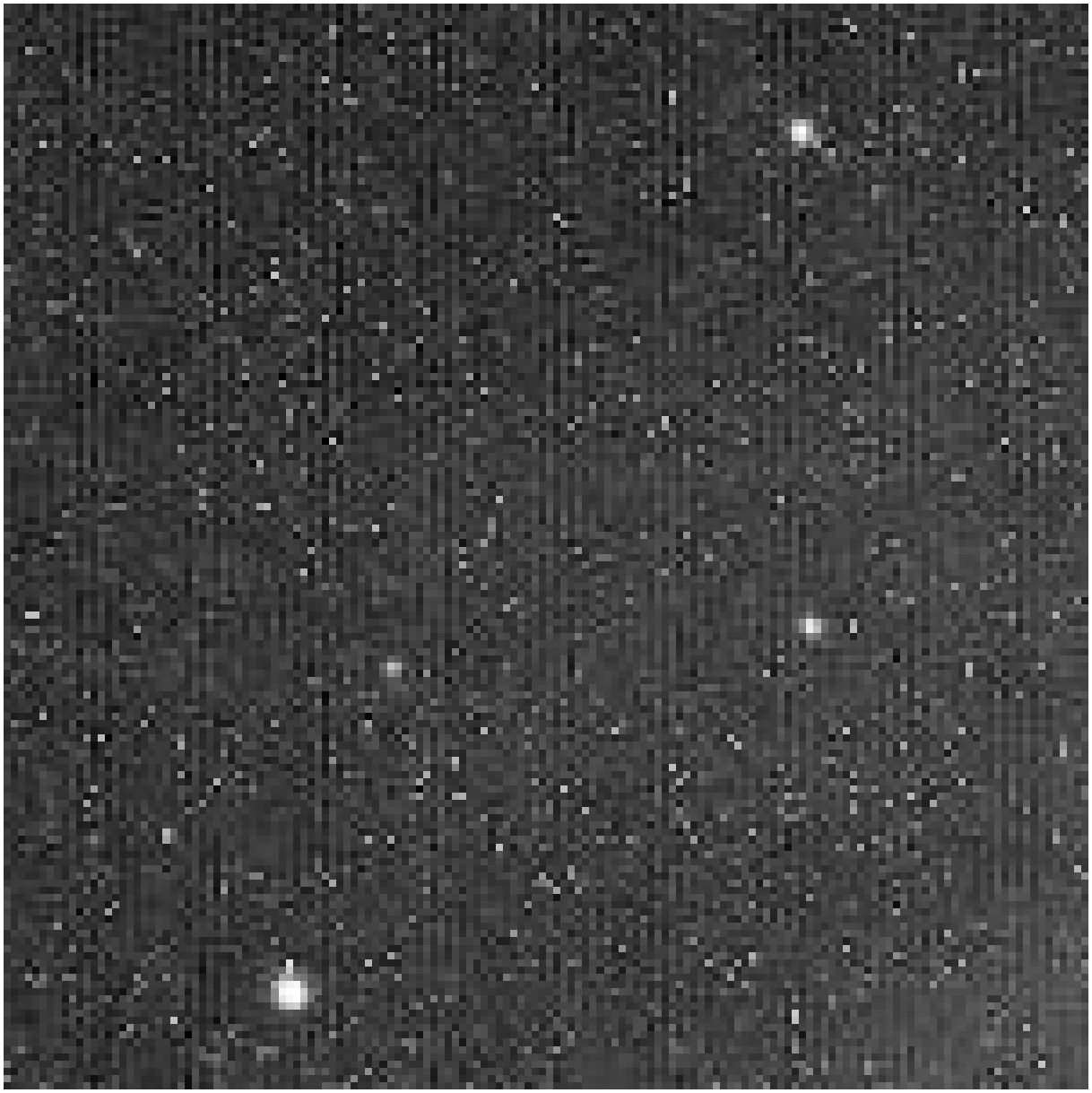


Fig 3.2. Zoom imagen ideal con padding size = (50,50)

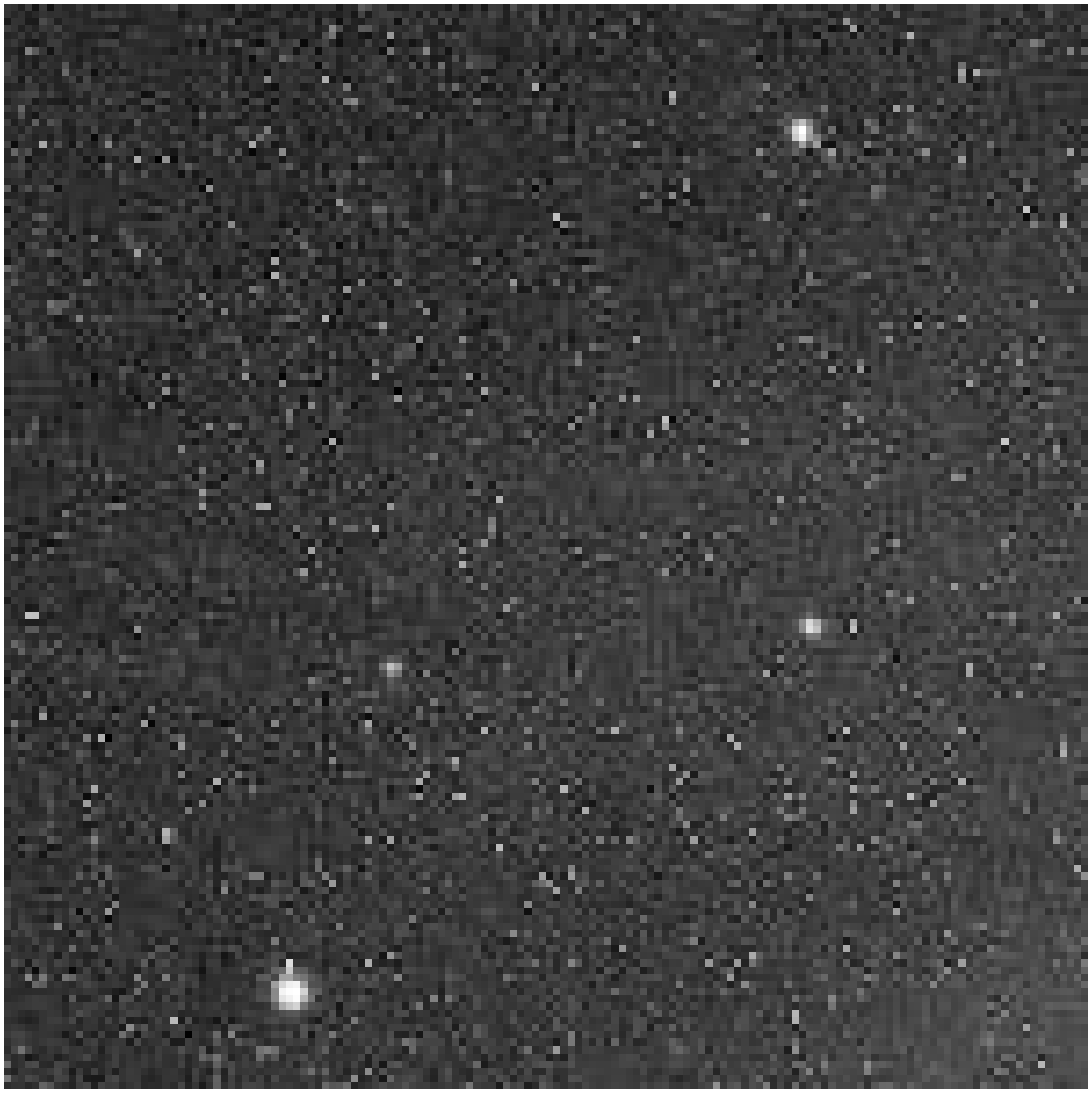


Fig 3.3. Zoom imagen ideal con padding size = (h/2, l/2)

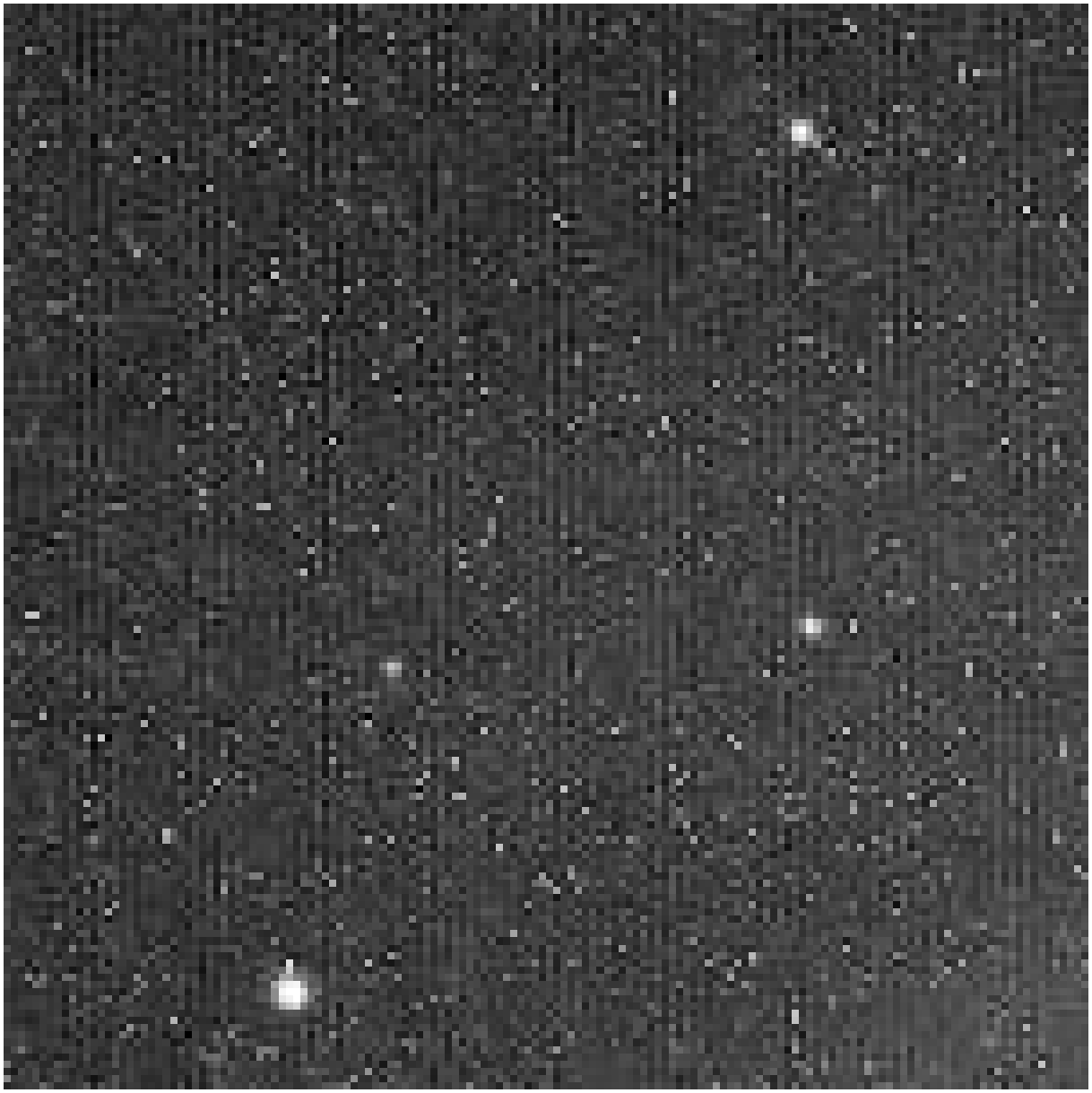


Fig 3.4. Zoom imagen ideal sin padding

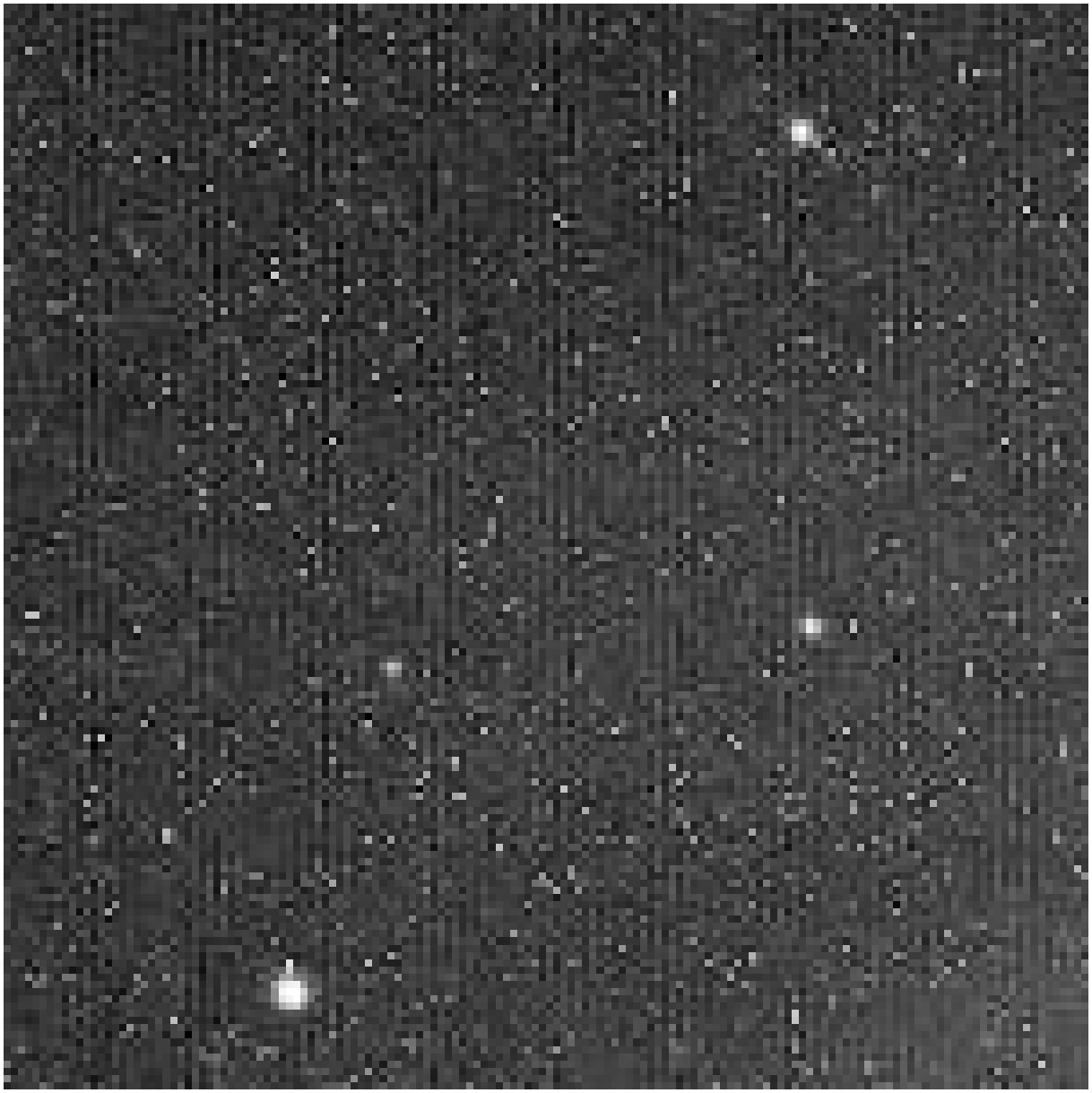


Fig 3.5. Zoom imagen ideal con padding size = (50,50)

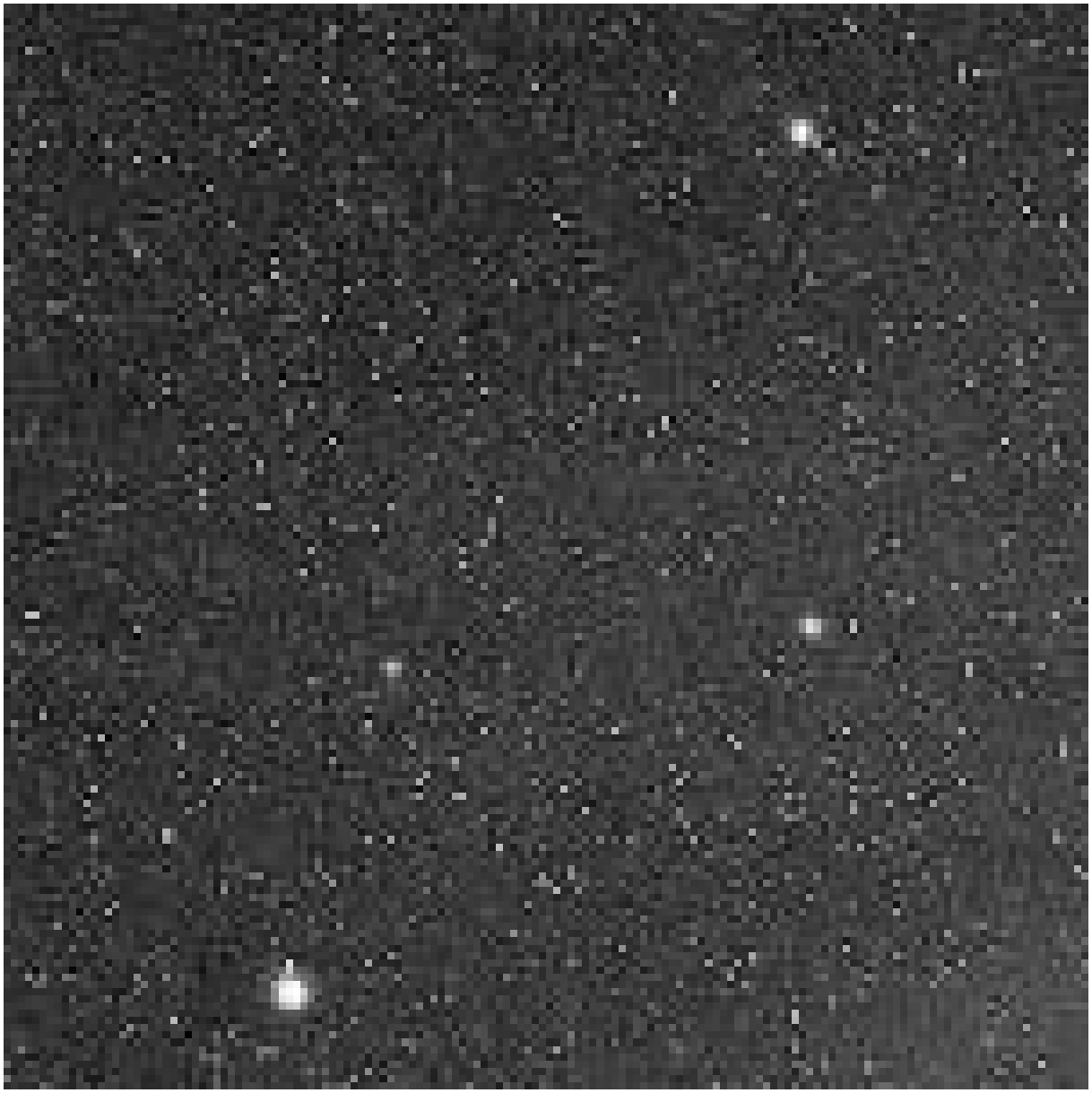


Fig 3.6. Zoom imagen ideal con padding size = (h/2, l/2)



Fig 4.1. Zoom imagen ideal normal vs sin simetría



Fig 4.2. Zoom imagen gauss normal vs sin simetría

Pregunta 2:

Fig 1.

Referencias: